

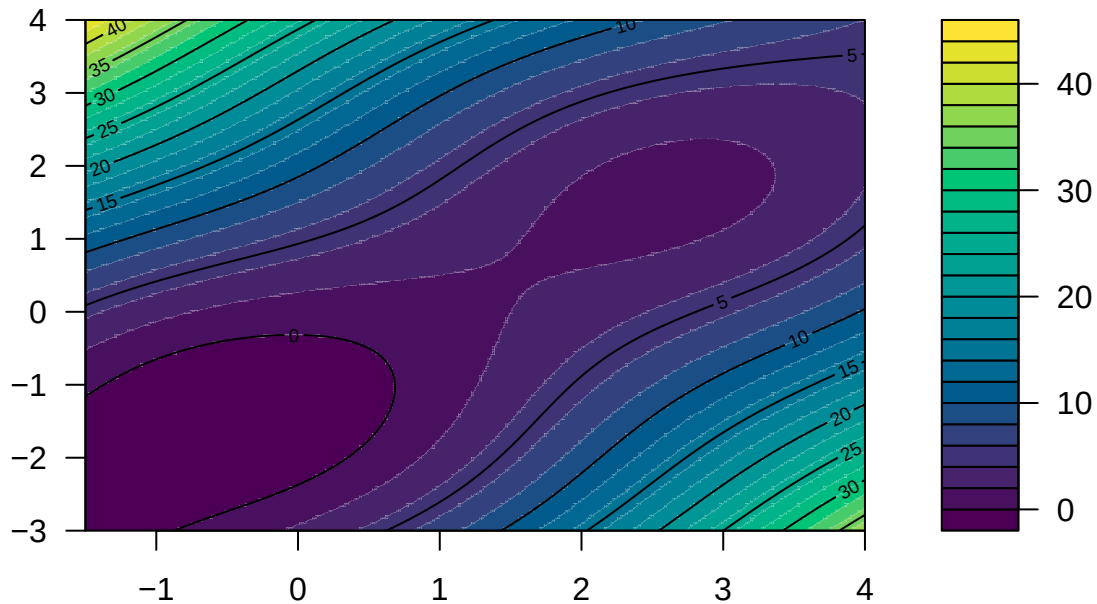
Lab 2

Aron Enström (aroen488), Sergey Volchkov (servo519)

2026-01-28

Q1: Optimization of a two-dimensional function

1.1. Function plot



1.2. Gradient and Hessian

For

$$f(x, y) = \sin(x + y) + (x - y)^2 - 1.5x + 2.5y + 1,$$

the gradient is

$$\nabla f(x, y) = \begin{pmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{pmatrix} = \begin{pmatrix} \cos(x + y) + 2(x - y) - 1.5 \\ \cos(x + y) - 2(x - y) + 2.5 \end{pmatrix}.$$

The Hessian matrix (matrix of second partial derivatives) is

$$\nabla^2 f(x, y) = \begin{pmatrix} \frac{\partial^2 f}{\partial x^2} & \frac{\partial^2 f}{\partial x \partial y} \\ \frac{\partial^2 f}{\partial y \partial x} & \frac{\partial^2 f}{\partial y^2} \end{pmatrix} = \begin{pmatrix} -\sin(x+y) + 2 & -\sin(x+y) - 2 \\ -\sin(x+y) - 2 & -\sin(x+y) + 2 \end{pmatrix},$$

where $\frac{\partial^2 f}{\partial x \partial y} = \frac{\partial^2 f}{\partial y \partial x}$.

1.3 Newton algorithm

We are asked to search for the global minimum solution within the box

$$x \in [-1.5, 4], \quad y \in [-3, 4].$$

so the algorithm code needs to be adjusted to prevent the search path wandering outside this box.

Compared to the original (unconstrained, “pure”) Newton algorithm, we introduced three standard safeguards and one additional fallback so the method behaves well on a bounded, potentially nonconvex objective.

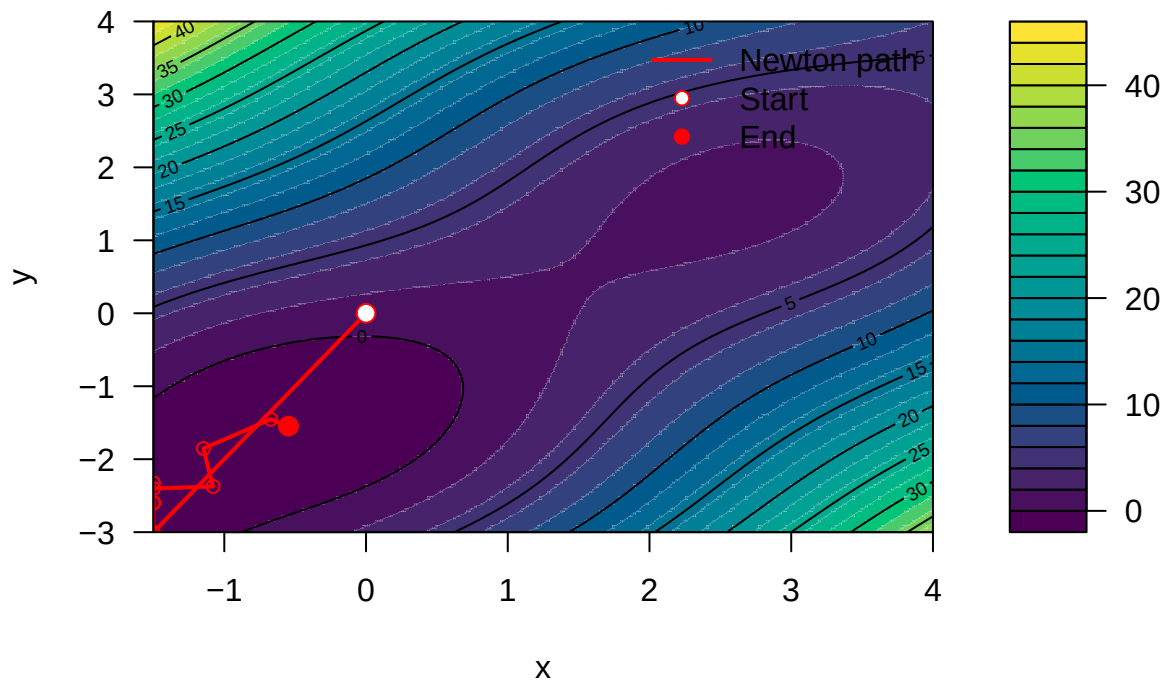
First, we enforced the box constraints by *projecting* every proposed iterate back into the feasible region: after forming a candidate update, we replace it by its componentwise clipped version so that $x \in [-1.5, 4]$ and $y \in [-3, 4]$ always hold.

Second, we stabilized the Newton step computation by *regularizing* the Hessian. Instead of solving $H(x_k, y_k) p_k = \nabla f(x_k, y_k)$ (which can fail when H is singular or ill-conditioned), we solve $(H(x_k, y_k) + \lambda I) p_k = \nabla f(x_k, y_k)$ and increase λ as needed until the linear system is solvable.

Third, we added a *backtracking line search* (step damping). Rather than always taking a full Newton step (i.e., step size $\alpha = 1$), we update via $(x_{k+1}, y_{k+1}) = P((x_k, y_k) - \alpha p_k)$ (where $P(\cdot)$ denotes the projection operator) and reduce α (e.g., $\alpha \leftarrow \beta \alpha$ with $0 < \beta < 1$) until an Armijo-type sufficient decrease condition is satisfied, which prevents overshooting and improves robustness on nonconvex landscapes.

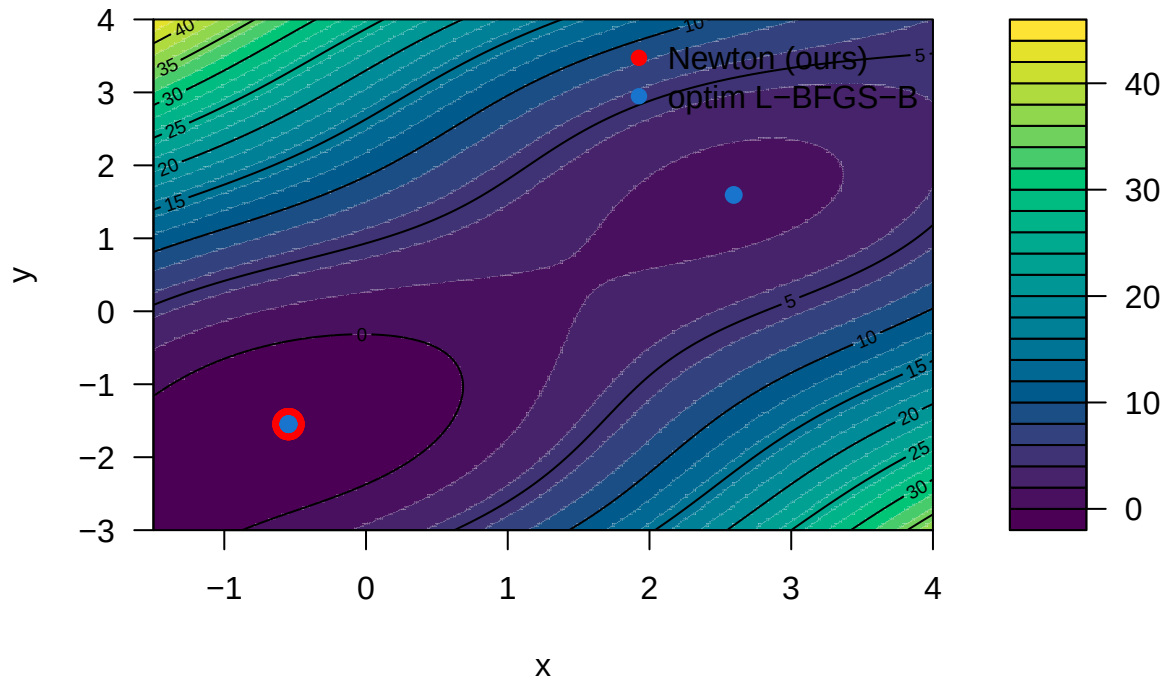
Finally, to avoid stalling at the boundary (where projection can collapse the effective Newton step and force α to become extremely small), we included a fallback to a *projected-gradient* step when the Newton step fails to produce progress. Concretely, we switch to $(x_{k+1}, y_{k+1}) = P((x_k, y_k) - \alpha \nabla f(x_k, y_k))$, which provides a feasible descent direction and helps the algorithm move along edges instead of getting trapped at corners.

Testing convergence from starting point at (0,0)



Comparing with the results produced by L-BFGS-B algorithm (BGFS with constraints) on a small sample of points.

Comparing the results with L-BFGS-B algorithm on a sample of points

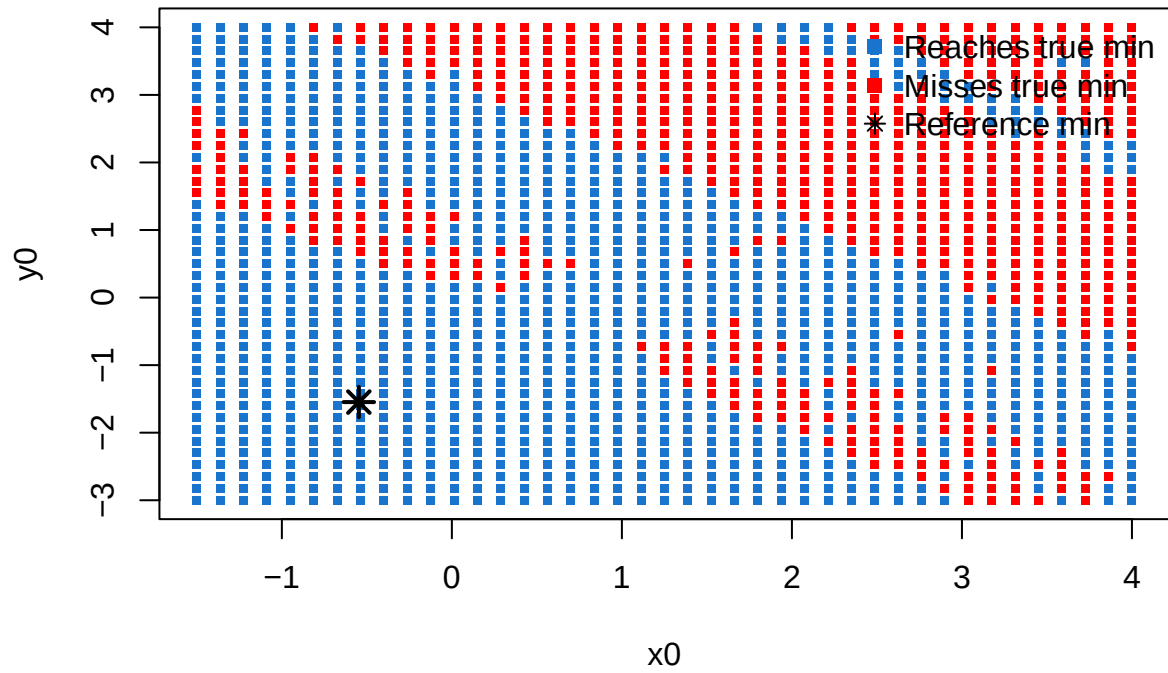


We can see from the above diagram that our algorithm converges in the same spot from all the randomly selected starting points, the endpoint seems to be where we expect the global minimum to lie on our heatmap plot and it coincides with the one produced by the L-BFGS-B algorithm which gives us confidence that it is working correctly.

1.4 Testing with different starting points

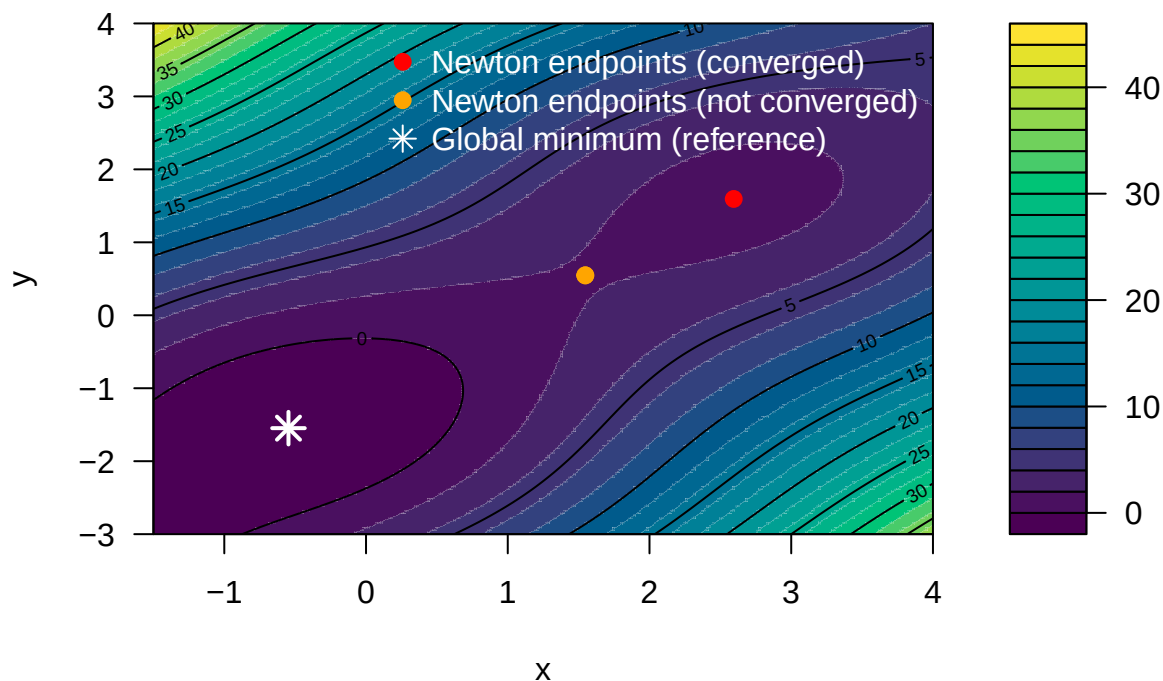
To identify the points from which the algorithms does not converge to the true minimum, we need to use a denser grid:

Starts: Newton misses (red) vs reaches (blue) the true minimum



Using the above output to see which points the algorithm converges to when it does not converge to the true minimum:

Attractors other than the global minimum



1.5 Understanding the nature of the incorrect endpoints

```
##
##      Global minimum      Local min Saddle / indefinite
##              1              1              2
##
##      end_x    end_y    end_f converged      class
## 1  -0.5471976 -1.547198 -1.913223      1      Global minimum
## x   2.5943950  1.594395  1.228370      1          Local min
## x77 1.5471670  0.547160  1.913223      0 Saddle / indefinite
## x91 1.5471450  0.547145  1.913223      0 Saddle / indefinite
```

Q2: Maximum likelihood

2.1. Steepest ascent function

See code in the appendix.

2.2. Computing the ML-estimator using two different backtracking methods

```
##      method      beta0      beta1    logLik iters n_g n_grad
## 1 increase -0.009354850 1.262812 -6.484279   41  59   41
## 2  armijo -0.009355336 1.262808 -6.484279   40  58   40
```

where `n_g` is the number of function evaluations and `n_grad` is the number of gradient evaluations.

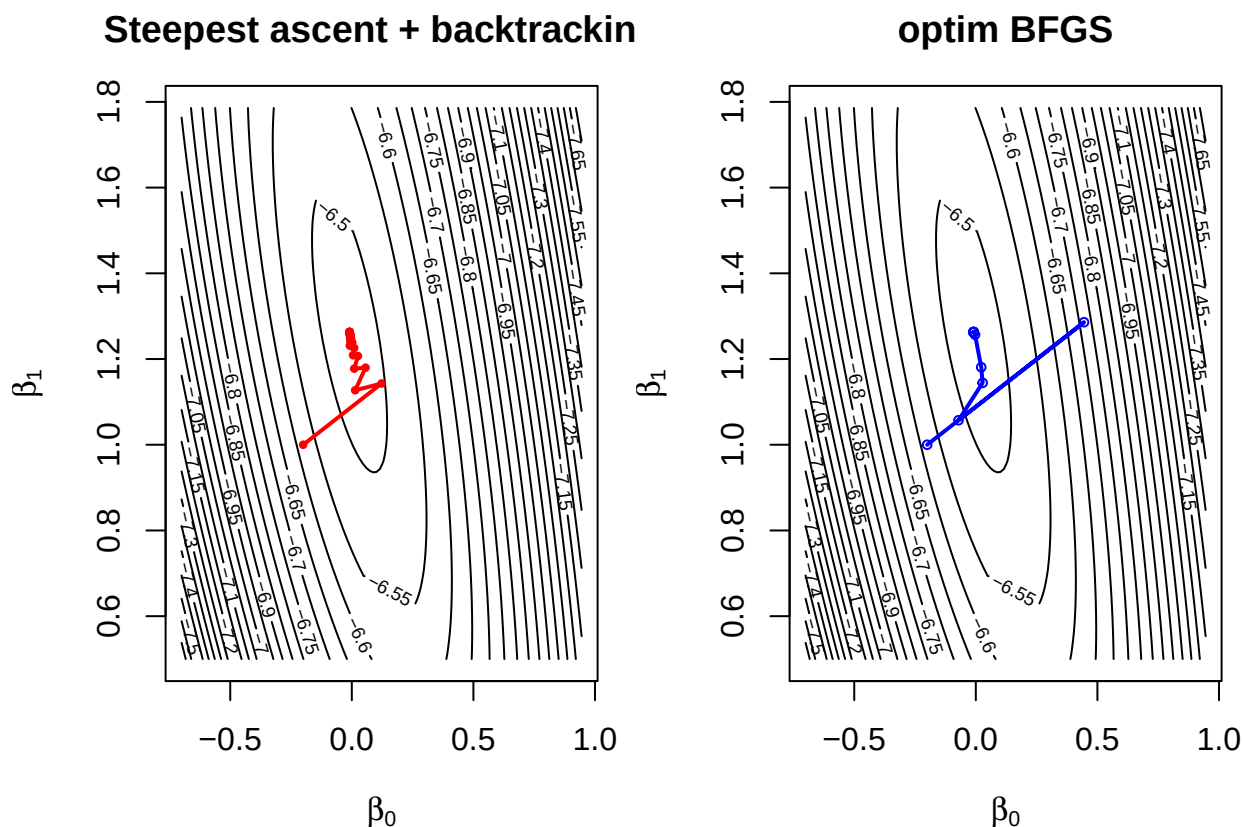
We have implemented two different backtracking line search methods: increase-only backtracking (“increase”) and Armijo backtracking (“armijo”). Both methods choose a step size $\alpha > 0$ for the update $b_{new} = b + \alpha d$,

where in steepest ascent we used the direction $d = \nabla g(b)$. The increase-only method accepts the first tried α (starting from α_0 and shrinking, e.g. $\alpha \leftarrow \rho\alpha$ with $0 < \rho < 1$) such that the log-likelihood does not decrease: $g(b + \alpha d) \geq g(b)$. This ensures monotone improvement, but it does not require the improvement to be “large enough” relative to what the local linear model predicts. Armijo accepts the first α (again shrinking from α_0) such that a *sufficient increase* condition holds: $g(b + \alpha d) \geq g(b) + c\alpha \nabla g(b)^T d$, $c \in (0, 1)$. So Armijo compares the realized improvement to the predicted first-order improvement and requires at least a fraction c of it. Armijo is typically stricter, it may backtrack more and thus use more function evaluations, especially when c is not tiny.

2.3. Comparing with optim() output

##	method	beta0	beta1	logLik	fn_evals	gr_evals	conv
## 1	BFGS	-0.009356126	1.262813	-6.484279	12	8	0
## 2	Nelder-Mead	-0.009423433	1.262738	-6.484279	47	NA	0

We’re observing slight differences in precision. Compared to our implementation, BFGS does significantly fewer function and gradient evaluations. BFGS needs far fewer evaluations because it uses curvature information to choose much better search directions than steepest ascent, so it typically reaches the optimum in many fewer outer iterations and often with fewer backtracking rejections per iteration. Nelder-Mead does more function evaluations but it does not calculate gradients.



2.4. Comparing with glm() output

##	method	beta0	beta1	logLik
## 1	glm (binomial)	-0.009359853	1.262823	-6.484279

The results seem to match the ones obtained with our function and optim().

Appendix

```
#1.1
f <- function(x, y) sin(x + y) + (x - y)^2 - 1.5*x + 2.5*y + 1

x <- seq(-1.5, 4, length.out = 300)
y <- seq(-3, 4, length.out = 300)

z <- outer(x, y, Vectorize(f))

filled.contour(x, y, z,
  nlevels = 30,
  color.palette = hcl.colors,
  plot.axes = {
    axis(1); axis(2)
    contour(x, y, z, add = TRUE, drawlabels = TRUE)
  }
)

#1.2
## Gradient
grad_f <- function(x, y) {
  c(
    dx = cos(x + y) + 2*(x - y) - 1.5,
    dy = cos(x + y) - 2*(x - y) + 2.5
  )
}

## Hessian
hess_f <- function(x, y) {
  s <- -sin(x + y)
  matrix(
    c(s + 2, s - 2,
      s - 2, s + 2),
    nrow = 2, byrow = TRUE,
    dimnames = list(c("x", "y"), c("x", "y"))
  )
}

## wrappers that take par = c(x, y) for optimization
fn_par <- function(par) f(par[1], par[2])
gr_par <- function(par) grad_f(par[1], par[2])
hess_par <- function(par) hess_f(par[1], par[2])

## test
# par0 <- c(0, 0)
# fn_par(par0)
# gr_par(par0)
# hess_par(par0)
#1.3

newton2d_box <- function(fn, gr, hess, x0, y0,
  lower, upper,
  tol = 1e-8, maxit = 200,
  lambda0 = 1e-6, lambda_mult = 10,
```



```

        alpha0 = 1, beta = 0.5, c1 = 1e-4,
        pg_alpha0 = 1, pg_beta = 0.5,
        eps_active = 1e-10,
        ..., verbose = FALSE, keep_path = TRUE) {

project <- function(par) pmin(pmax(par, lower), upper)

## projected-gradient infinity norm (KKT-style for box constraints)
projgrad_inf <- function(x, g, eps = 1e-12) {
  pg <- g
  for (i in 1:2) {
    if (x[i] <= lower[i] + eps) pg[i] <- min(g[i], 0)
    if (x[i] >= upper[i] - eps) pg[i] <- max(g[i], 0)
  }
  max(abs(pg))
}

## "gradient mapping" direction:  $d = P(x - a g) - x$  (feasible descent step)
gradmap_dir <- function(x, g, a) project(x - a * g) - x

x <- project(c(x0, y0))
f0 <- fn(x[1], x[2], ...)

path <- NULL
if (keep_path) {
  path <- data.frame(iter = 0L, x = x[1], y = x[2], f = f0,
                    step = NA_real_, method = "init",
                    projgrad = projgrad_inf(x, gr(x[1], x[2], ...)))
}

for (k in 1:maxit) {
  g <- gr(x[1], x[2], ...)
  pg_norm <- projgrad_inf(x, g)

  if (pg_norm < tol) {
    return(list(par = x, value = f0, iter = k - 1, converged = TRUE,
              projgrad = pg_norm, path = path))
  }

  ## ---- Attempt Newton step (with regularization + projected Armijo) ----
  H <- hess(x[1], x[2], ...)
  lambda <- lambda0
  p <- NULL
  repeat {
    p_try <- tryCatch(solve(H + lambda * diag(2), g), error = function(e) NULL)
    if (!is.null(p_try)) { p <- p_try; break }
    lambda <- lambda * lambda_mult
    if (lambda > 1e12) { p <- NULL; break }
  }

  newton_ok <- FALSE
  a <- alpha0
  if (!is.null(p)) {

```

```

repeat {
  x_new <- project(x - a * p)
  f_new <- fn(x_new[1], x_new[2], ...)
  armijo_rhs <- f0 - c1 * a * sum(g * p) # sufficient decrease
  if (is.finite(f_new) && f_new <= armijo_rhs && any(abs(x_new - x) > 0)) {
    newton_ok <- TRUE
    break
  }
  a <- a * beta
  if (a < 1e-12) break
}

if (newton_ok) {
  x <- x_new; f0 <- f_new
  if (keep_path) path <- rbind(path, data.frame(iter = k, x = x[1], y = x[2], f = f0,
                                                step = a, method = "Newton",
                                                projgrad = pg_norm))
  if (verbose) cat(sprintf("iter=%d Newton  x=%.8f y=%.8f f=%.10f step=%.1e ||pg||_inf=%.3e\n",
                           k, x[1], x[2], f0, a, pg_norm))
  next
}

## ---- Fallback: projected-gradient step (guarantees feasible movement) ----
a_pg <- pg_alpha0
repeat {
  d <- gradmap_dir(x, g, a_pg)
  x_new <- x + d
  f_new <- fn(x_new[1], x_new[2], ...)
  if (is.finite(f_new) && f_new <= f0) break # simple monotone decrease
  a_pg <- a_pg * pg_beta
  if (a_pg < 1e-12) break
}

## If even PG can't decrease, stop (we're at a constrained stationary-ish point)
if (sqrt(sum((x_new - x)^2)) < tol) {
  return(list(par = x, value = f0, iter = k, converged = FALSE,
             projgrad = pg_norm, path = path,
             message = "No progress (Newton+PG failed to decrease)."))
}

x <- x_new; f0 <- f_new
if (keep_path) path <- rbind(path, data.frame(iter = k, x = x[1], y = x[2], f = f0,
                                              step = a_pg, method = "PG",
                                              projgrad = pg_norm))
if (verbose) cat(sprintf("iter=%d PG      x=%.8f y=%.8f f=%.10f step=%.1e ||pg||_inf=%.3e\n",
                           k, x[1], x[2], f0, a_pg, pg_norm))
}

list(par = x, value = f0, iter = maxit, converged = FALSE,
     projgrad = projgrad_inf(x, gr(x[1], x[2], ...)),
     path = path)
}

```

```

# newton2d_box <- function(fn, gr, hess, x0, y0,
#                           lower, upper,
#                           tol = 1e-8, maxit = 100,
#                           lambda0 = 1e-6, lambda_mult = 10,
#                           alpha0 = 1, beta = 0.5, c1 = 1e-4,
#                           eps_active = 1e-10,
#                           ..., verbose = FALSE, keep_path = TRUE) {
#
#   project <- function(par) pmin(pmax(par, lower), upper)
#
#   projgrad_inf <- function(x, g, eps = 1e-12) {
#     pg <- g
#     for (i in 1:2) {
#       if (x[i] <= lower[i] + eps) pg[i] <- min(g[i], 0)
#       if (x[i] >= upper[i] - eps) pg[i] <- max(g[i], 0)
#     }
#     max(abs(pg))
#   }
#
#   ## Active-set: free if not too close to either bound
#   free_set <- function(x) {
#     c(
#       (x[1] > lower[1] + eps_active) && (x[1] < upper[1] - eps_active),
#       (x[2] > lower[2] + eps_active) && (x[2] < upper[2] - eps_active)
#     )
#   }
#
#   x <- project(c(x0, y0))
#   f0 <- fn(x[1], x[2], ...)
#
#   path <- NULL
#   if (keep_path) {
#     path <- data.frame(iter = 0L, x = x[1], y = x[2], f = f0,
#                       step = NA_real_, projgrad = NA_real_, lambda = NA_real_,
#                       free_x = NA, free_y = NA)
#   }
#
#   for (k in 1:maxit) {
#     g <- gr(x[1], x[2], ...)
#     H <- hess(x[1], x[2], ...)
#     pg_norm <- projgrad_inf(x, g)
#
#     free <- free_set(x)
#     if (keep_path) {
#       path <- rbind(path, data.frame(iter = k, x = x[1], y = x[2], f = f0,
#                                     step = NA_real_, projgrad = pg_norm, lambda = NA_real_,
#                                     free_x = free[1], free_y = free[2]))
#     }
#
#     if (pg_norm < tol) {
#       return(list(par = x, value = f0, iter = k - 1, converged = TRUE,
#                  projgrad = pg_norm, path = path))
#     }
#   }

```

```

#
#   ## If nothing is free, we're at a corner/edge; stop (or switch to PG)
#   if (!any(free)) {
#       return(list(par = x, value = f0, iter = k - 1, converged = FALSE,
#                   projgrad = pg_norm, path = path,
#                   message = "Stuck at corner: no free variables."))
#   }
#
#   ## Reduced Newton direction p: only update free components
#   p_full <- c(0, 0)
#   idx <- which(free)
#
#   lambda <- lambda0
#   repeat {
#       Hred <- H[idx, idx, drop = FALSE]
#       gred <- g[idx]
#       p_try <- tryCatch(solve(Hred + lambda * diag(length(idx)), gred),
#                         error = function(e) NULL)
#       if (!is.null(p_try)) { p_full[idx] <- p_try; break }
#       lambda <- lambda * lambda_mult
#       if (lambda > 1e12) stop("Could not compute reduced Newton step.")
#   }
#
#   ## Line search on reduced step (projection just guards numerics)
#   a <- alpha0
#   repeat {
#       x_new <- project(x - a * p_full)
#       f_new <- fn(x_new[1], x_new[2], ...)
#
#       armijo_rhs <- f0 - c1 * a * sum(g[idx] * p_full[idx]) # uses free components
#       if (is.finite(f_new) && f_new <= armijo_rhs) break
#
#       a <- a * beta
#       if (a < 1e-12) break
#   }
#
#   if (verbose) {
#       cat(sprintf("iter=%d x=%.8f y=%.8f f=%.10f step=%.1e ||pg||_inf=%.3e lambda=%.1e free=(%d
#                   k, x_new[1], x_new[2], f_new, a, pg_norm, lambda, free[1], free[2]))
#   }
#
#   if (keep_path) {
#       path[nrow(path), "step"] <- a
#       path[nrow(path), "lambda"] <- lambda
#   }
#
#   ## Secondary stop: tiny move AND small projected grad
#   if (sqrt(sum((x_new - x)^2)) < tol && pg_norm < tol) {
#       return(list(par = x_new, value = f_new, iter = k, converged = TRUE,
#                   projgrad = pg_norm, path = path))
#   }
#
#   ## If the step collapsed, bail out instead of looping forever

```

```

#   if (a < 1e-12) {
#       return(list(par = x, value = f0, iter = k, converged = FALSE,
#                   projgrad = pg_norm, path = path,
#                   message = "Line search failed (step underflow)."))
#   }
#
#   x <- x_new
#   f0 <- f_new
# }
#
# list(par = x, value = f0, iter = maxit, converged = FALSE,
#      projgrad = projgrad_inf(x, gr(x[1], x[2], ...)),
#      path = path)
# }

# newton2d_box <- function(fn, gr, hess, x0, y0,
#                          lower, upper,
#                          tol = 1e-8, maxit = 100,
#                          lambda0 = 1e-6, lambda_mult = 10,
#                          alpha0 = 1, beta = 0.5, c1 = 1e-4,
#                          ..., verbose = FALSE, keep_path = TRUE) {
#
#   project <- function(par) pmin(pmax(par, lower), upper)
#
#   ## infinity-norm of projected gradient (KKT-style for box constraints)
#   projgrad_inf <- function(x, g, eps = 1e-12) {
#     pg <- g
#     for (i in 1:2) {
#       if (x[i] <= lower[i] + eps) pg[i] <- min(g[i], 0)
#       if (x[i] >= upper[i] - eps) pg[i] <- max(g[i], 0)
#     }
#     max(abs(pg))
#   }
#
#   x <- project(c(x0, y0))
#   f0 <- fn(x[1], x[2], ...)
#
#   ## store path (x, y, f, step, projgrad, lambda)
#   path <- NULL
#   if (keep_path) {
#     path <- data.frame(iter = 0L, x = x[1], y = x[2], f = f0,
#                        step = NA_real_, projgrad = NA_real_, lambda = NA_real_)
#   }
#
#   for (k in 1:maxit) {
#     g <- gr(x[1], x[2], ...)
#     pg_norm <- projgrad_inf(x, g)
#
#     if (keep_path) path <- rbind(path, data.frame(
#       iter = k, x = x[1], y = x[2], f = f0,
#       step = NA_real_, projgrad = pg_norm, lambda = NA_real_
#     ))
#   }

```

```

#   if (pg_norm < tol) {
#       return(list(par = x, value = f0, iter = k - 1, converged = TRUE,
#                   projgrad = pg_norm, path = path))
#   }
#
#   H <- hess(x[1], x[2], ...)
#   I2 <- diag(2)
#
#   ## Regularize Hessian until solvable
#   lambda <- lambda0
#   p <- NULL
#   repeat {
#       p_try <- tryCatch(solve(H + lambda * I2, g), error = function(e) NULL)
#       if (!is.null(p_try)) { p <- p_try; break }
#       lambda <- lambda * lambda_mult
#       if (lambda > 1e12) stop("Could not compute Newton step (even after regularization).")
#   }
#
#   ## Backtracking line search with Armijo sufficient decrease
#   a <- alpha0
#   repeat {
#       x_new <- project(x - a * p)
#       f_new <- fn(x_new[1], x_new[2], ...)
#
#       armijo_rhs <- f0 - c1 * a * sum(g * p)
#
#       if (is.finite(f_new) && f_new <= armijo_rhs) break
#       a <- a * beta
#       if (a < 1e-12) break
#   }
#
#   if (verbose) {
#       cat(sprintf(
#           "iter=%d x=%.8f y=%.8f f=%.10f step=%.1e ||pg||_inf=%.3e lambda=%.1e\n",
#           k, x_new[1], x_new[2], f_new, a, pg_norm, lambda
#       ))
#   }
#
#   if (keep_path) {
#       path[nrow(path), "step"] <- a
#       path[nrow(path), "lambda"] <- lambda
#       path[nrow(path), "projgrad"] <- pg_norm
#   }
#
#   ## Only use step-size convergence as a secondary criterion
#   if (sqrt(sum((x_new - x)^2)) < tol && pg_norm < tol) {
#       return(list(par = x_new, value = f_new, iter = k, converged = TRUE,
#                   projgrad = pg_norm, path = path))
#   }
#
#   x <- x_new
#   f0 <- f_new
# }

```

```

#
#   list(par = x, value = f0, iter = maxit, converged = FALSE,
#         projgrad = projgrad_inf(x, gr(x[1], x[2], ...)),
#         path = path)
# }

# newton2d_box <- function(fn, gr, hess, x0, y0,
#                           lower, upper,
#                           tol = 1e-8, maxit = 100,
#                           lambda0 = 1e-6, lambda_mult = 10,
#                           alpha0 = 1, beta = 0.5, c1 = 1e-4,
#                           ..., verbose = FALSE) {
#
#   project <- function(par) pmin(pmax(par, lower), upper)
#
#   ## infinity-norm of projected gradient (KKT-style for box constraints)
#   projgrad_inf <- function(x, g, eps = 1e-12) {
#     pg <- g
#     for (i in 1:2) {
#       if (x[i] <= lower[i] + eps) pg[i] <- min(g[i], 0) # should be >=0 at lower bound
#       if (x[i] >= upper[i] - eps) pg[i] <- max(g[i], 0) # should be <=0 at upper bound
#     }
#     max(abs(pg))
#   }
#
#   x <- project(c(x0, y0))
#   f0 <- fn(x[1], x[2], ...)
#
#   for (k in 1:maxit) {
#     g <- gr(x[1], x[2], ...)
#     pg_norm <- projgrad_inf(x, g)
#
#     if (pg_norm < tol) {
#       return(list(par = x, value = f0, iter = k - 1, converged = TRUE,
#                   projgrad = pg_norm))
#     }
#
#     H <- hess(x[1], x[2], ...)
#     I2 <- diag(2)
#
#     ## Regularize Hessian until solvable
#     lambda <- lambda0
#     p <- NULL
#     repeat {
#       p_try <- tryCatch(solve(H + lambda * I2, g), error = function(e) NULL)
#       if (!is.null(p_try)) { p <- p_try; break }
#       lambda <- lambda * lambda_mult
#       if (lambda > 1e12) stop("Could not compute Newton step (even after regularization).")
#     }
#
#     ## Backtracking line search with Armijo sufficient decrease
#     a <- alpha0
#     repeat {

```

```

#     x_new <- project(x - a * p)
#     f_new <- fn(x_new[1], x_new[2], ...)
#
#     ## directional derivative along -p is approx -g^T p
#     armijo_rhs <- f0 - c1 * a * sum(g * p)
#
#     if (is.finite(f_new) && f_new <= armijo_rhs) break
#     a <- a * beta
#     if (a < 1e-12) break
#   }
#
#   if (verbose) {
#     cat(sprintf(
#       "iter=%d x=%.8f y=%.8f f=%.10f step=%.1e ||pg||_inf=%.3e lambda=%.1e\n",
#       k, x_new[1], x_new[2], f_new, a, pg_norm, lambda
#     ))
#   }
#
#   ## Only use step-size convergence as a secondary criterion
#   if (sqrt(sum((x_new - x)^2)) < tol && pg_norm < tol) {
#     return(list(par = x_new, value = f_new, iter = k, converged = TRUE,
#               projgrad = pg_norm))
#   }
#
#   x <- x_new
#   f0 <- f_new
# }
#
# list(par = x, value = f0, iter = maxit, converged = FALSE,
#      projgrad = projgrad_inf(x, gr(x[1], x[2], ...)))
# }

# newton2d_box <- function(fn, gr, hess, x0, y0,
#                          lower, upper,
#                          tol = 1e-8, maxit = 100,
#                          lambda0 = 1e-6, lambda_mult = 10,
#                          alpha0 = 1, beta = 0.5,
#                          ..., verbose = FALSE) {
#
#   #To keep the results within the prescribed x,y limits
#   project <- function(par) pmin(pmax(par, lower), upper)
#
#   x <- project(c(x0, y0))
#   f0 <- fn(x[1], x[2], ...)
#
#   for (k in 1:maxit) {
#     g <- gr(x[1], x[2], ...)
#     H <- hess(x[1], x[2], ...)
#     I2 <- diag(2)
#
#     ## regularize until solvable
#     lambda <- lambda0

```



```

#   p <- NULL
#   repeat {
#     p_try <- tryCatch(solve(H + lambda * I2, g), error = function(e) NULL)
#     if (!is.null(p_try)) { p <- p_try; break }
#     lambda <- lambda * lambda_mult
#     if (lambda > 1e12) stop("Could not compute Newton step.")
#   }
#
#   ## backtracking line search with projection
#   a <- alpha0
#   repeat {
#     x_new <- project(x - a * p)
#     f_new <- fn(x_new[1], x_new[2], ...)
#     if (is.finite(f_new) && f_new <= f0) break
#     a <- a * beta
#     if (a < 1e-12) break
#   }
#
#   if (verbose) {
#     cat(sprintf("iter=%d  x=%.8f y=%.8f  f=%.10f  step=%.1e\n",
#                 k, x_new[1], x_new[2], f_new, a))
#   }
#
#   if (sqrt(sum((x_new - x)^2)) < tol) {
#     return(list(par = x_new, value = f_new, iter = k, converged = TRUE))
#   }
#
#   x <- x_new
#   f0 <- f_new
# }
#
#   list(par = x, value = f0, iter = maxit, converged = FALSE)
# }

# ##  $x_{k+1} = x_k - H(x_k)^{-1} * g(x_k)$ 
# newton2d_reg <- function(fn, gr, hess, x0, y0,
#                           tol = 1e-8, maxit = 100,
#                           lambda0 = 1e-6, lambda_mult = 10,
#                           ..., verbose = FALSE) {
#
#   x <- c(x0, y0)
#
#   for (k in 1:maxit) {
#     g <- gr(x[1], x[2], ...)
#     H <- hess(x[1], x[2], ...)
#     I2 <- diag(2)
#
#     lambda <- lambda0
#     p <- NULL
#
#     ## Keep increasing lambda until the linear system is solvable
#     repeat {
#       Hp <- H + lambda * I2

```

```

#       p_try <- tryCatch(solve(Hp, g), error = function(e) NULL) # catch singularity
#       if (!is.null(p_try)) { p <- p_try; break }
#       lambda <- lambda * lambda_mult
#       if (lambda > 1e12) stop("Hessian remained singular/ill-conditioned even after regularization.")
#     }
#
#     x_new <- x - p
#
#     if (verbose) {
#       cat(sprintf("iter=%d  x=%.10f y=%.10f  f=%.10f  ||grad||=%.3e  lambda=%.1e\n",
#                   k, x_new[1], x_new[2], fn(x_new[1], x_new[2], ...),
#                   sqrt(sum(g^2)), lambda))
#     }
#
#     if (sqrt(sum((x_new - x)^2)) < tol) {
#       return(list(par = x_new, value = fn(x_new[1], x_new[2], ...),
#                  iter = k, converged = TRUE))
#     }
#     x <- x_new
#   }
#
#   list(par = x, value = fn(x[1], x[2], ...), iter = maxit, converged = FALSE)
# }
#
lower <- c(-1.5, -3)
upper <- c( 4.0,  4)
# res <- newton2d_box(fn = f, gr = grad_f, hess = hess_f,
#                    x0 = 0, y0 = 0, lower=lower, upper=upper, verbose = TRUE)
# res$par
# res$value
#
#
res00 <- newton2d_box(fn = f, gr = grad_f, hess = hess_f,
                     x0 = 0, y0 = 0,
                     lower = lower, upper = upper,
                     verbose = FALSE, keep_path = TRUE)

# head(res00$path)

filled.contour(x, y, z,
  nlevels = 30,
  color.palette = hcl.colors,
  plot.title = title(main = "Testing convergence from starting point at (0,0)",
                     xlab = "x", ylab = "y"),
  plot.axes = {
    axis(1); axis(2)
    contour(x, y, z, add = TRUE, drawlabels = TRUE)

    ## Newton path: line + points (start = open circle, end = filled circle)
    lines(res00$path$x, res00$path$y, col = "red", lwd = 2)
    points(res00$path$x, res00$path$y, col = "red", pch = 1, cex = 0.9)
    points(res00$path$x[1], res00$path$y[1], col = "red", pch = 21, bg = "white", cex = 1.3)
  }

```

```

points(tail(res00$path$x, 1), tail(res00$path$y, 1), col = "red", pch = 19, cex = 1.3)

legend("topright",
      legend = c("Newton path", "Start", "End"),
      col = c("red", "red", "red"),
      lwd = c(2, NA, NA),
      pch = c(NA, 21, 19),
      pt.bg = c(NA, "white", NA),
      bty = "n")
}
)
lower <- c(-1.5, -3)
upper <- c( 4.0,  4)

one_run <- function(x0, y0) {
  nres <- newton2d_box(f, grad_f, hess_f, x0, y0, lower, upper)

  ores <- optim(
    par = c(x0, y0),
    fn = fn_par, gr = gr_par,
    method = "L-BFGS-B",
    lower = lower, upper = upper
  ) # box constraints supported by L-BFGS-B

  data.frame(
    x0 = x0, y0 = y0,
    newton_iter = nres$iter,
    newton_x = nres$par[1], newton_y = nres$par[2],
    newton_value = nres$value,
    optim_iter = ores$counts[["function"]],
    optim_x = ores$par[1], optim_y = ores$par[2],
    optim_value = ores$value
  )
}

starts <- rbind(
  c(-1.0, -2.0),
  c( 3.5,  3.5),
  c( 0.0,  0.0),
  c( 4.0, -3.0),
  c(-1.5,  4.0)
)
colnames(starts) <- c("x0", "y0")

res_table <- do.call(
  rbind,
  lapply(seq_len(nrow(starts)), function(i) one_run(starts[i, 1], starts[i, 2]))
)

res_table_fmt <- res_table
num_cols <- names(res_table_fmt)[sapply(res_table_fmt, is.numeric)]

```

```

res_table_fmt[num_cols] <- lapply(res_table_fmt[num_cols], \(v) formatC(v, format = "f", digits = 6))
# print(res_table_fmt, row.names = FALSE)

## distinct solutions reached (rounded to group near-equal points)
# unique(round(res_table[, c("newton_x", "newton_y")], 6))
# unique(round(res_table[, c("optim_x", "optim_y")], 6))

filled.contour(x, y, z,
  nlevels = 30,
  color.palette = hcl.colors,
  plot.title = title(main = "Comparing the results with L-BFGS-B algorithm\non a sample of points",
    xlab = "x", ylab = "y"),
  plot.axes = {
    axis(1); axis(2)
    contour(x, y, z, add = TRUE, drawlabels = TRUE)

    ## Overlay solutions
    points(res_table$newton_x, res_table$newton_y,
      pch = 19, col = "red", cex = 2.1)
    points(res_table$optim_x, res_table$optim_y,
      pch = 19, col = "dodgerblue3", cex = 1.1)

    legend("topright",
      legend = c("Newton (ours)", "optim L-BFGS-B"),
      col = c("red", "dodgerblue3"),
      pch = 19, bty = "n")
  }
)
#1.4
lower <- c(-1.5, -3)
upper <- c( 4.0,  4)

## multi-start L-BFGS-B to approximate the global minimum in the box
x_st <- seq(lower[1], upper[1], length.out = 15)
y_st <- seq(lower[2], upper[2], length.out = 15)
starts_ref <- expand.grid(x0 = x_st, y0 = y_st) # all combinations

run_lbfgsb <- function(x0, y0) {
  o <- optim(par = c(x0, y0),
    fn = fn_par, gr = gr_par,
    method = "L-BFGS-B",
    lower = lower, upper = upper) # bound constraints
  list(par = o$par, value = o$value, counts = o$counts)
}

ref_list <- lapply(seq_len(nrow(starts_ref)),
  function(i) run_lbfgsb(starts_ref$x0[i], starts_ref$y0[i]))
ref_vals <- sapply(ref_list, `[[`, "value")
best_i <- which.min(ref_vals)

x_star_ref <- ref_list[[best_i]]$par
f_star_ref <- ref_list[[best_i]]$value

```

```

# x_star_ref; f_star_ref

tol_val <- 1e-6

## grid (or increase resolution / use random points)
xg <- seq(lower[1], upper[1], length.out = 41)
yg <- seq(lower[2], upper[2], length.out = 41)
starts <- expand.grid(x0 = xg, y0 = yg)

one_run_newton <- function(x0, y0) {
  nres <- newton2d_box(f, grad_f, hess_f, x0, y0, lower, upper)

  data.frame(
    x0 = x0, y0 = y0,
    converged = isTRUE(nres$converged),
    iters = nres$iter,
    x = nres$par[1], y = nres$par[2],
    f = nres$value,
    miss_true_min = (!isTRUE(nres$converged)) || (nres$value > f_star_ref + tol_val)
  )
}

scan_res <- do.call(rbind, lapply(seq_len(nrow(starts)),
                                function(i) one_run_newton(starts$x0[i], starts$y0[i])))

## starting points where Newton does NOT reach the (estimated) global min
bad_starts <- subset(scan_res, miss_true_min)

# nrow(bad_starts)
# head(bad_starts)

cols <- ifelse(scan_res$miss_true_min, "red", "dodgerblue3")
plot(scan_res$x0, scan_res$y0, col = cols, pch = 15, cex = 0.6,
     xlab = "x0", ylab = "y0",
     main = "Starts: Newton misses (red) vs reaches (blue) the true minimum")
points(x_star_ref[1], x_star_ref[2], pch = 8, cex = 1.5, lwd = 2)
legend("topright", legend = c("Reaches true min", "Misses true min", "Reference min"),
     col = c("dodgerblue3", "red", "black"), pch = c(15, 15, 8), bty = "n")

endpts <- do.call(rbind, lapply(seq_len(nrow(bad_starts)), function(i) {
  x0 <- bad_starts$x0[i]
  y0 <- bad_starts$y0[i]

  r <- newton2d_box(f, grad_f, hess_f, x0, y0, lower, upper, keep_path = TRUE)

  data.frame(
    x0 = x0, y0 = y0,
    end_x = r$par[1],
    end_y = r$par[2],
    end_f = r$value,
    converged = isTRUE(r$converged),
    iter = r$iter
  )
}))

```

```

}))

endpts<-unique(round(endpts[, c("end_x","end_y","end_f", "converged")], 6))

filled.contour(x, y, z,
  nlevels = 30,
  color.palette = hcl.colors,
  plot.title = title(main = "Attractors other than the global minimum",
    xlab = "x", ylab = "y"),
  plot.axes = {
    axis(1); axis(2)
    contour(x, y, z, add = TRUE, drawlabels = TRUE)

    ## Endpoints of Newton from the bad starts
    cols <- ifelse(endpts$converged, "red", "orange")
    points(endpts$end_x, endpts$end_y, pch = 19, col = cols, cex = 1.1)

    ## Global minimum (reference)
    points(x_star_ref[1], x_star_ref[2], pch = 8, col = "white", cex = 1.6, lwd = 2)

    legend("topright",
      legend = c("Newton endpoints (converged)",
        "Newton endpoints (not converged)",
        "Global minimum (reference)"),
      pch = c(19, 19, 8),
      col = c("red", "orange", "white"),
      pt.cex = c(1.1, 1.1, 1.3),
      text.col = "white",
      bty = "n")
  }
)

#1.4
# Since none of the detected endpoints lies near the boundary, we can assume it is not
# due to the boundary interfering with the workings of our algorithm
# We can therefore classify the endpoints by eigenvalues of  $H(x,y)$ 
# - all eigenvalues  $>0$  -> local minimum
# - mixed signs -> saddle
# - all  $<0$  -> local maximum
classify_interior <- function(x, y, grad_f, hess_f, tol_g = 1e-6) {
  g <- grad_f(x, y)
  H <- hess_f(x, y)
  eg <- eigen(H, symmetric = TRUE)$values

  # if (sqrt(sum(g^2)) > tol_g) return("Not stationary (interior)")

  if (all(eg > 0)) "Local min"
  else if (all(eg < 0)) "Local max"
  else "Saddle / indefinite"
}

endpts$class <- apply(endpts[, c("end_x", "end_y")], 1, \v) {
  classify_interior(v[1], v[2], grad_f, hess_f)
})

```

```

global_row <- data.frame(
  end_x = x_star_ref[1],
  end_y = x_star_ref[2],
  end_f = f(x_star_ref[1], x_star_ref[2]),
  class = "Global minimum",
  converged = TRUE
)

endpts2 <- rbind(endpts, global_row)

table(endpts2$class)

endpts2[order(endpts2$end_f), ]
#2.1

# Steepest ascent with backtracking line search for ML estimation
steepestasc_logit_count <- function(
  x, y,
  b0 = c(0, 0),
  alpha0 = 1,
  shrink = 0.5,
  maxit = 5000,
  max_backtrack = 60,
  tol_par = 0.5e-5,
  tol_rel = NULL,
  tol_grad = NULL,
  ls_type = c("increase", "armijo"),
  armijo_c = 1e-4,
  trace = FALSE
) {
  ls_type <- match.arg(ls_type)

  n_g <- 0L
  n_gr <- 0L

  g_logit <- function(b) {
    n_g <- n_g + 1L
    eta <- b[1] + b[2] * x
    sum(y * eta - log1p(exp(eta)))
  }

  grad_logit <- function(b) {
    n_gr <- n_gr + 1L
    eta <- b[1] + b[2] * x
    p <- 1 / (1 + exp(-eta))
    r <- y - p
    c(sum(r), sum(r * x))
  }

  b <- b0
  ll <- g_logit(b)

  for (it in 1:maxit) {

```

```

gr <- grad_logit(b)
if (!all(is.finite(gr))) stop("Non-finite gradient encountered.")

d <- gr # steepest ascent direction

alpha <- alpha0
b_new <- b + alpha * d
ll_new <- g_logit(b_new)

bt <- 0L
if (ls_type == "increase") {
  while (ll_new < ll && bt < max_backtrack) {
    alpha <- alpha * shrink
    b_new <- b + alpha * d
    ll_new <- g_logit(b_new)
    bt <- bt + 1L
  }
} else { # Armijo sufficient increase for maximization
  rhs <- ll + armijo_c * alpha * sum(gr * d)
  while (ll_new < rhs && bt < max_backtrack) {
    alpha <- alpha * shrink
    b_new <- b + alpha * d
    ll_new <- g_logit(b_new)
    rhs <- ll + armijo_c * alpha * sum(gr * d)
    bt <- bt + 1L
  }
}

if (bt >= max_backtrack) {
  warning("Backtracking failed to find an acceptable step; stopping.")
  break
}

step <- b_new - b

stop_par_abs <- max(abs(step)) < tol_par
stop_par_rel <- FALSE
if (!is.null(tol_rel)) {
  stop_par_rel <- max(abs(step) / pmax(1, abs(b_new))) < tol_rel
}
stop_grad <- FALSE
if (!is.null(tol_grad)) {
  stop_grad <- max(abs(gr)) < tol_grad
}

if (trace) {
  cat(sprintf(
    "it=%d ll=%.10f alpha=%.4g step_inf=%.3e b=(%.6f, %.6f) ls=%s\n",
    it, ll_new, alpha, max(abs(step)), b_new[1], b_new[2], ls_type
  ))
}

b <- b_new

```



```

    ll <- ll_new

    if (stop_par_abs || stop_par_rel || stop_grad) break
  }

  list(beta = b, logLik = ll, iterations = it, n_g = n_g, n_grad = n_gr, ls_type = ls_type)
}
#2.2

x <- c(0,0,0, 0.1,0.1, 0.3,0.3, 0.9,0.9,0.9)
y <- c(0,0,1, 0,1, 1,1, 0,1,1)
b0=c(-0.2,1)
methods <- c("increase", "armijo")
res_list <- vector("list", length(methods))

for (k in seq_along(methods)) {
  m <- methods[k]
  if (m == "armijo") {
    res_list[[k]] <- steepestasc_logit_count(x, y, b0=b0, ls_type=m, armijo_c=0.1)
  } else {
    res_list[[k]] <- steepestasc_logit_count(x, y, b0=b0, ls_type=m)
  }
}

results <- data.frame(
  method = methods,
  beta0 = sapply(res_list, function(r) r$beta[1]),
  beta1 = sapply(res_list, function(r) r$beta[2]),
  logLik = sapply(res_list, function(r) r$logLik),
  iters = sapply(res_list, function(r) r$iterations),
  n_g = sapply(res_list, function(r) r$n_g),
  n_grad = sapply(res_list, function(r) r$n_grad)
)

results
#2.3
# We maximize logLik, but optim minimizes -> use neg logLik and its gradient
fn_negll <- function(b, x, y) {
  eta <- b[1] + b[2] * x
  -sum(y * eta - log1p(exp(eta)))
}

gr_negll <- function(b, x, y) {
  eta <- b[1] + b[2] * x
  p <- 1 / (1 + exp(-eta))
  r <- y - p
  -c(sum(r), sum(r * x))
}

methods <- c("BFGS", "Nelder-Mead")
res <- vector("list", length(methods))

for (k in seq_along(methods)) {

```

```

m <- methods[k]

if (m == "BFGS") {
  # BFGS can use the supplied gradient
  res[[k]] <- optim(
    par = b0,
    fn = fn_negll, gr = gr_negll,
    x = x, y = y,
    method = m
  )
} else {
  # Nelder-Mead is derivative-free; no gradient evaluations
  res[[k]] <- optim(
    par = b0,
    fn = fn_negll,
    x = x, y = y,
    method = m,
    control = list(warn.1d.NelderMead = FALSE)
  )
}
}

tbl <- data.frame(
  method = methods,
  beta0 = sapply(res, function(r) r$par[1]),
  beta1 = sapply(res, function(r) r$par[2]),
  logLik = sapply(res, function(r) -r$value),
  fn_evals = sapply(res, function(r) unname(r$counts[["function"]])),
  gr_evals = sapply(res, function(r) unname(r$counts[["gradient"]])),
  conv = sapply(res, function(r) r$convergence)
)

tbl
steepestasc_logit_path <- function(
  x, y, b0=c(0,0),
  alpha0=1, shrink=0.5, maxit=5000, max_backtrack=60,
  tol_par=0.5e-5,
  ls_type=c("increase","armijo"), armijo_c=1e-4
) {
  ls_type <- match.arg(ls_type)

  g_logit <- function(b) {
    eta <- b[1] + b[2]*x
    sum(y*eta - log1p(exp(eta)))
  }
  grad_logit <- function(b) {
    eta <- b[1] + b[2]*x
    p <- 1/(1+exp(-eta))
    r <- y - p
    c(sum(r), sum(r*x))
  }

  b <- b0

```

```

ll <- g_logit(b)

path <- matrix(NA_real_, nrow=maxit+1, ncol=2)
path[1,] <- b

for (it in 1:maxit) {
  gr <- grad_logit(b)
  d <- gr

  alpha <- alpha0
  b_new <- b + alpha*d
  ll_new <- g_logit(b_new)

  bt <- 0L
  if (ls_type == "increase") {
    while (ll_new < ll && bt < max_backtrack) {
      alpha <- alpha*shrink
      b_new <- b + alpha*d
      ll_new <- g_logit(b_new)
      bt <- bt + 1L
    }
  } else {
    rhs <- ll + armijo_c*alpha*sum(gr*d)
    while (ll_new < rhs && bt < max_backtrack) {
      alpha <- alpha*shrink
      b_new <- b + alpha*d
      ll_new <- g_logit(b_new)
      rhs <- ll + armijo_c*alpha*sum(gr*d)
      bt <- bt + 1L
    }
  }
  if (bt >= max_backtrack) break

  step <- b_new - b
  b <- b_new
  ll <- ll_new
  path[it+1,] <- b

  if (max(abs(step)) < tol_par) break
}

path <- path[complete.cases(path), , drop=FALSE]
list(beta=b, logLik=ll, path=path)
}

# log-likelihood and negative for optim (optim minimizes)
g_logit <- function(b, x, y) {
  eta <- b[1] + b[2]*x
  sum(y*eta - log1p(exp(eta)))
}
fn_negll <- function(b, x, y) -g_logit(b, x, y)
gr_negll <- function(b, x, y) {
  eta <- b[1] + b[2]*x

```

```

p <- 1/(1+exp(-eta))
r <- y - p
-c(sum(r), sum(r*x))
}

# Wrapper to record path
bfgs_with_path <- function(b0, x, y) {
  path <- matrix(numeric(0), ncol=2)
  fn_wrap <- function(b, x, y) {
    path <- rbind(path, b)
    fn_negll(b, x, y)
  }
  res <- optim(par=b0, fn=fn_wrap, gr=gr_negll, x=x, y=y, method="BFGS")
  list(beta=res$par, logLik=-res$value, path=path)
}

sa <- steepestasc_logit_path(x, y, b0=b0, ls_type="armijo", armijo_c=1e-4)
bf <- bfgs_with_path(b0, x, y)

# Make a grid around the solution area
all_pts <- rbind(sa$path, bf$path)
b0_rng <- range(all_pts[,1])
b1_rng <- range(all_pts[,2])

b0_seq <- seq(b0_rng[1]-0.5, b0_rng[2]+0.5, length.out=120)
b1_seq <- seq(b1_rng[1]-0.5, b1_rng[2]+0.5, length.out=120)

Z <- outer(b0_seq, b1_seq, Vectorize(function(a0, a1) g_logit(c(a0,a1), x, y)))

op <- par(mfrow=c(1,2), mar=c(4,4,3,1))

contour(b0_seq, b1_seq, Z, nlevels=25,
        xlab=expression(beta[0]), ylab=expression(beta[1]),
        main="Steepest ascent + backtracking")
lines(sa$path[,1], sa$path[,2], col="red", lwd=2)
points(sa$path[,1], sa$path[,2], col="red", pch=16, cex=0.6)

contour(b0_seq, b1_seq, Z, nlevels=25,
        xlab=expression(beta[0]), ylab=expression(beta[1]),
        main="optim BFGS")
lines(bf$path[,1], bf$path[,2], col="blue", lwd=2)
points(bf$path[,1], bf$path[,2], col="blue", pch=1, cex=0.6)

par(op)
#2.4

dat <- data.frame(y=y, x=x)
fit_glm <- glm(y ~ x, family = binomial(link = "logit"), data = dat)

b <- coef(fit_glm)
ll <- as.numeric(logLik(fit_glm))

glm_tbl <- data.frame(

```

```
method = "glm (binomial)",  
beta0 = unname(b[1]),  
beta1 = unname(b[2]),  
logLik = ll  
)  
glm_tbl
```